

前端构建工具

1. 课程目标

- 掌握前端构建工具的发展史；
- 掌握常见的前端构建工具；

2. 课程大纲

1. 构建工具发展史；
2. 构建工具详解；

3. 构建工具发展史

构建其实是工程化、自动化思想在前端开发中的体现（个人认为狭义的工程化就是构建工具+CI/CD），将一系列流程用代码去实现，让代码自动化地执行这一系列复杂的流程。构建为前端开发注入了更大的活力，解放了我们的生产力。

说到前端的构建工具，不得不从前端模块化说起（参考：[前端模块化](#)）

以下分别标识出前端模块工具的发展历程（主要参考常见的构建工具，不完善请补充）：

1. 2009 年，Kevin Dangoor 发起了 ServerJS 项目，后更名为 CommonJS，其目标是指定浏览器外的 JS API 规范（例如 FS、Stream、Buffer 等）以及模块规范 Modules/1.0。这一规范也成为同年发布的 NodeJS 中的模块定义的参照规范；
2. 2011 年，RequireJS 1.0 版本发布，作为客户端的模块加载器，提供了异步加载模块的能力。作者在之后提交了 CommonJS 的 Module/Transfer/C 提案，这一提案最终发展为了独立的 AMD 规范；
3. 2013 年，Grunt、Gulp 第一版相继发布，同年，面向浏览器端模块的打包工具 Browserify 发布；
4. 2014 年，跨平台的前后端兼容的模块化定义语法 UMD 发布；
5. 2014 年，Sebastian McKenzie 发布了将 ES6 语法转换为 ES5 语法的工具 6to5，并在之后更名为 Babel；
6. 2014 年，Guy Bedford 对外发布了 SystemJS 和 jspm 工具，用于简化模块加载和处理包管理；

7. 2014 年，打包工具 Webpack 发布了第一个稳定版本；
8. 2015 年，ES6 (ES2015) 规范正式发布，第一次从语言规范上定义了 JS 中的模块化；
9. 2015 年，Rich Harris 发布的 Rollup 项目，基于 ES6 模块化，提供了 Tree Shaking 的功能；
10. 2017年，鉴于当时的Webpack使用上过于繁琐，官方文档也不是很清晰明了。所以Parcel一发布就被推上了风口浪尖；
11. 2019年，snowpack将node_modules转为ESM的构建工具开始出现；
12. 2020年，随着浏览器对ESM和HTTP2的支持，bundleless思路也开始出现，esbuild此时开始出现在视野里，同时snowpack也开始内置使用esbuild；
13. 2021年，Vite出现；

由此可见，随着前端技术的演进，整体的构建工具也随之演进，后文主要对目前为止比较流行的构建工具进行分析，主要包括以下几类：

1. 初版构建工具；
2. 现代打包构建工具基石--webpack；
3. 基于webpack改进的构建工具；
4. 突破JS语言特性的构建工具；
5. 基于ES Module的bundleless构建工具；

4. 构建工具详解

4.1. 初版构建工具

4.1.1. Grunt

Grunt可以说是前端（JS语言）中第一个正式的构建工具，它基于 Node.js ，用 JS 开发，这样就可以借助 Node.js 实现跨系统跨平台的桌面端的操作，例如文件操作等等。此外，Grunt 以及插件都作为一个包，可以用 NPM 安装进行管理，但对于像webpack上很多能力，如HMR、Scope Hoist等都是不支持的，可以作为学习webpack前的了解。

Grunt其实更像是一种自动化的配置工具集，就像官方说的，Grunt是 The JavaScript Task Runner，使用上，Grunt 是由 JSON 配置设置驱动，并且每个 Grunt 任务通常必须创建中间文件将结果传递给其他任务。

所能实现的功能包括：检查每个 JS 文件语法、合并两个 JS 文件、将合并后的 JS 文件压缩、将 SCSS 文件编译等，包括我们上面提到的试用gulp将.less文件转换为.css文件等等。

- [grunt plugins](#)

4.1.2. Gulp

Gulp跟Grunt类似，都是基于task驱动执行的，可以完成javascript/coffee/sass/less/html/image/css等文件的测试、检查、合并、压缩、格式化、浏览器自动刷新、部署文件生成。Gulp的优点在于Gulp更倾向于写代码的方式，但相关的插件资源不如Grunt。

同样的，现代的构建工具基本不使用Gulp了，有兴趣看官网自行学习，这里不详细介绍。

- [Gulp Plugins](#)

4.2. 现代打包构建工具基石--webpack

详情见webpack详解。

4.3. 基于webpack改进的构建工具

4.3.1. rollup

虽然rollup与webpack一样，都是通过解析JavaScript的依赖树将代码输出为指定版本的JavaScript，供浏览器或者node环境执行。不同的是rollup相对webpack更轻量，其构建的代码并不会像webpack一样被注入大量的webpack内部结构，而是尽可能的精简保持代码原有的状态。且配置相对简单，但因为没有devServer和HMR，所以一般rollup用于JS库开发，而非业务开发。

rollup生成代码只是把我们的代码转码成目标JS，同时如果需要,他可以同时帮我们生成支持umd/commonjs/es的js代码，vue/react/angular都在用他作为打包工具。查看他们的官网代码都可以看到rollup的影子。

注意：不要看rollup的中文官网，时效性太差，很多新特性都不维护了。

4.3.1.1. rollup使用

浏览器环境使用的应用程序的话：

1. 无需考虑浏览器兼容问题

- a. 开发者写esm代码 -> rollup通过入口，递归识别esm模块 -> 最终打包成一个或多个bundle.js

-> 浏览器直接可以支持引入 `<script type="module">`

2. 需考虑浏览器兼容问题

- a. 可能会比较复杂，需要用额外的polyfill库，或结合webpack使用

打包成npm包的话：

1. 开发者写esm代码 -> rollup通过入口，递归识别esm模块 -> （可以支持配置输出多种格式的模块，如esm、cjs、umd、amd）最终打包成一个或多个bundle.js
 - a. （开发者要写cjs也可以，需要插件@rollup/plugin-commonjs）；
 - b. 很明显，rollup 比较适合打包js库（react、vue等的源代码库都是rollup打包的），这样打包出来的库，可以充分使用上esm的tree shaking，使源库体积最小；

针对一个相同的demo，webpack和rollup打包出的体积相差极大：

webpack诞生于ESM标准出来前，CommonJs出来后，当时的浏览器只能通过script标签加载模块。script标签加载代码是没有作用域的，只能在代码内用iife的方式实现作用域效果，这就是webpack打包出来的代码大结构都是iife的原因，并且每个模块都要装到function里面，才能保证互相之间作用域不干扰。这就是为什么webpack打包的代码为什么乍看会感觉乱，找不到自己写的代码的真正原因。同时，webpack的代码注入问题，是因为浏览器不支持cjs，所以webpack要去自己实现require和module.exports方法（才有很多注入）（webpack自己实现polyfill）

那这么多年了，甚至到现在2022年，浏览器为什么不支持cjs？

1. cjs是同步的，运行时的，node环境用cjs，node本身运行在服务器，无需等待网络握手，所以同步处理是很快的；
2. 浏览器是客户端，访问的是服务端资源，中间需要等待网络握手，可能会很慢，所以不能同步的卡在那里等服务器返回的，体验太差；
3. 后续出来esm后，webpack为了兼容以前发在npm上的老包（并且当时心还不够决绝，导致这种“丑结构的包”越来越多，以后就更不可能改这种“丑结构了”），所以保留这个iife的结构和代码注入，导致现在看webpack打包的产物，乍看结构比较乱且有很多的代码注入，自己写的代码都找不到；

rollup诞生于esm标准出来后，就是针对esm设计的，也没有历史包袱，所以可以做到真正的“打包”（精简，无额外注入）

4.3.1.2. 总结

rollup在构建JavaScript方面比webpack有更大的优势：

1. 构建速度明显快于webpack；

2. 生成的代码量很小；
3. 配置方式其实非常简单；

不过在应用开发层面讲，如果开发一个Web应用webpack要比rollup有更大的优势，因为其天然继承了devServer以及hmr，这使得开发者可以快速的对应用进行调试开发，同时webpack自身庞大且成熟的生态体系也让他更加适合应用开发，所以最终总结的就是rollup更加适合JS库开发，而webpack更加适合应用开发。

4.3.2. Parcel

Parcel诞生有很强的历史背景，当时webpack上各种配置过于繁琐，且官方文档也不清晰，导致人们开始转向其他的打包工具，这时Parcel就诞生了。

注意：Parcel的中文文档落后太多了，基本Parcel2的版本都没有，所以建议看英文；

4.3.2.1. Parcel介绍

Parcel的特点：

- 完全零配置；
- 构建速度更快；
- 自动安装依赖，开发更加便捷；

相较于Webpack，Parcel是以assets方式组织的，assets 可以是任意文件，所以你可以构建任意文件。而在 webpack 中，必须是以 JS 为入口去组织其他文件，这算是一个体验上的升级。

同时，速度快也是Parcel的优势，原因主要在于Parcel支持多核（通过worker平行构建）和文件系统缓存（二次构建会快，使用C++缓存，效率更高，和 webpack 的 dll 异曲同工），不过目前Webpack也有多核处理loader和压缩的插件。关于 0 配置。ParcelJS 本身是 0 配置的，但 HTML、JS 和 CSS 分别是通过 posthtml、babel 和 postcss 处理的，所以我们得分别配 `.posthtmlrc`、`.babelrc` 和 `.postcssrc`。所以Parcel更适用于小型简单的项目，定制化需求高的还是建议使用webpack（毕竟社区资源丰富）；

4.3.2.2. 总结

Parcel只能说是时代的产物，Parcel的code splitting、HMR、sourcemap、publicPath、tree shaking、scope hoist、share module、UMD等基本的能力还是提供了的，而且官方也在不断的维护，建议简单项目可以尝试使用。

市场

- [awesome Parcel](#);
- [codesandbox](#);

4.4. 突破JS语言特性的构建工具

4.4.1. SWC (Speedy Web Compiler)

SWC虽然名义上是编译器，但它实际上是compiler + bundler，只不过目前的bundler功能还有待完善提高

4.4.1.1. SWC介绍

SWC意指 (Speedy Web Compiler) 快速的web编译器，是用RUST实现的。主要针对JS使用了多线程的能力。

我们知道，Webpack与Babel的性能瓶颈都在于JS语言，出现了go实现的Esbuild与Rus实现的SWC等工具，其中，SWC的目标则是替代babel，本身是为了[对标babel](#)进行设计的，可以看到大部分babel的功能SWC也对标实现了。

具体的配置内容跟babel类似，且在webpack中，也有类似 `babel-loader` 的 `swc-loader`，我们可以在根目录下设置`.swcrc`文件，可以指定常见的编译内容，浏览器的支撑、模块化、代码压缩以及打包，具体的配置[见链接](#)；

除此之外，swc还支持插件的形式（基本上babel的能力都支持），但整体来看，还是有很多功能正在路上，鉴于 `@babel/types` 类似能力的缺失、TS支持程度等问题，还是建议在生产环境再观察一段时间，后续有条件再使用。

4.4.1.2. SWC使用

swc 与 babel 一样，将命令行工具、编译核心模块分化为两个包。

- `@swc/cli` 类似于 `@babel/cli`;
- `@swc/core` 类似于 `@babel/core`;

▼

复制代码

```
1 npm i -D @swc/cli @swc/core
```

通过如下命令，可以将一个 ES6 的 JS 文件转化为 ES5。

```
1 npx swc source.js -o dist.js
2
3 const start = () => {
4   console.log('app started')
5 }
6
7 // 转为
8 var start = function() {
9   console.log("app started");
10 };
11
```

配置的格式

与babel类似，在根目录下使用.swcrc

swc 与 babel 一样，支持类似于 .babelrc 的配置文件：.swcrc，配置的格式为 JSON。

```
1  {
2    "$schema": "https://json.schemastore.org/swcrc",
3    "jsc": {
4      "parser": {
5        "syntax": "ecmascript", // 还支持TS
6        "jsx": false,
7        "dynamicImport": false,
8        "privateMethod": false,
9        "functionBind": false,
10       "exportDefaultFrom": false,
11       "exportNamespaceFrom": false,
12       "decorators": false,
13       "decoratorsBeforeExport": false,
14       "topLevelAwait": false,
15       "importMeta": false
16     },
17     "transform": null,
18     "target": "es5",
19     "loose": false,
20     "externalHelpers": false,
21     // Requires v1.2.50 or upper and requires target to be es2016 or upper
22     "keepClassNames": false
23   },
24   "minify": false
25 }
```

babel 的插件系统被 swc 整合成了 jsc.parser 内的配置，基本上大部分插件都能照顾到。而且，swc 还继承了压缩的能力，通过 minify 属性开启，jsc.minify 用于配置压缩相关的规则。

API的使用

导入 @swc/core 模块，可以在 node.js 中调用 api 直接进行代码的编译


```

1  import { readFileSync } from 'fs'
2  import { transform } from '@swc/core'
3
4  const run = async () => {
5      const code = readFileSync('./source.js', 'utf-8')
6      const result = await transform(code, {
7          filename: "source.js",
8      })
9      // 输出编译后代码
10     console.log(result.code)
11 }
12
13 run()

```

打包

打包的能力在V1中称为 `spack`，但在 V2 中将重命名为 `swcpack`。`spack.config.js` 将被 `swcpack.config.js` 取代，但目前还没发布V2，所以目前还是使用`spack`。

```

1  // spack.config.js
2  const { config } = require('@swc/core/spack')
3
4
5  module.exports = config({
6      entry: {
7          'web': __dirname + '/src/index.ts',
8      },
9      output: {
10         path: __dirname + '/lib',
11         name: 'index.js' // name可选
12     },
13     module: {},
14 });

```

其中，配置项为 `spack.config.js`，接受

- mode: production | debug | none，目前还没有使用；
- entry: 入口文件，可以指定文件或者文件夹映射；
- output: 出口文件；

引入的文件为：

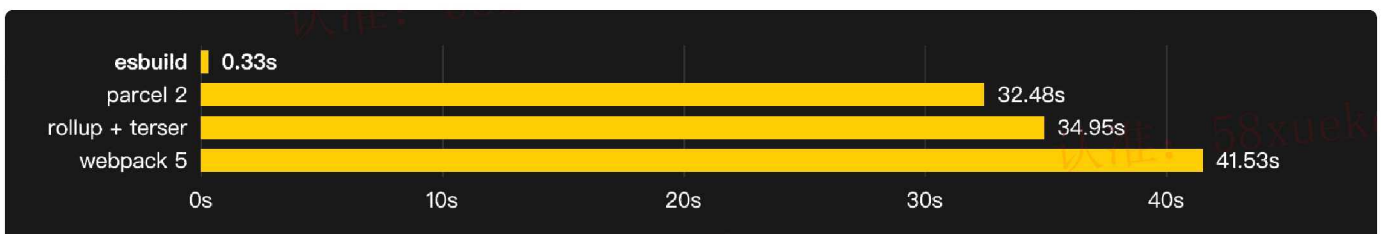
```
1 // index.ts
2 import { B } from './common';
3
4 console.log(B)
5
6 // common.ts
7 export const A = 'foo';
8 export const B = 'bar';
```

- example: [codesandbox](#);
- [在线编辑](#);

4.4.2. Esbuild

可能很多同学对于Esbuild的了解停留在Vite是使用Esbuild进行预构建依赖的，这也是为什么vite编译那么快的原因之一，下文将详细介绍下Esbuild的使用。

4.4.2.1. Esbuild为什么快



1. 使用Go

JavaScript 本质上依然是一门解释型语言，JavaScript 程序每次执行都需要先由解释器一边将源码翻译成机器语言，一边调度执行；而 Go 是一种编译型语言，在编译阶段就已经将源码转译为机器码，启动时只需要直接执行这些机器码即可。也就意味着，Go 语言编写的程序比 JavaScript 少了一个动态解释的过程。

2. 多线程

Go 天生具有多线程运行能力，对打包过程的解析、代码生成阶段进行了深度定制，而 JavaScript 本质上是一门单线程语言，直到引入 WebWorker 规范之后才有可能在浏览器、Node 中实现多线程操作，除了 CPU 指令运行层面的并行外，Go 语言多个线程之间还能共享相同的内存空间，而 JavaScript 的每个线程都有自己独有的内存堆。这意味着 Go 中多个处理单元，例如解析资源 A 的线程，可以直接读取资源 B 线程的运行结果，而在 JavaScript 中相同的操作需要调用通讯接口 `woker.postMessage` 在线程间复制数据。

所以在运行时层面，Go 拥有天然的多线程能力，更高效的内存使用率，也就意味着更高的运行性能。

3. 全量定制

在 Webpack、Rollup 这类工具中，比如：

- 使用 babel 实现 ES 版本转译；
- 使用 eslint 实现代码检查；
- 使用 TSC 实现 ts 代码转译与代码检查；
- 使用 less、stylus、sass 等 css 预处理工具；

这些全部都是插件实现，其实目前我们已经完全习惯了这种方式，甚至觉得事情就应该是这样的，大多数人可能根本没有意识到事情可以有另一种解决方案。Esbuild 起了个头，选择完全重写，整套编译流程所需要用到的所有工具全部重写，这意味着它需要重写 js、ts、jsx、json 等资源文件的加载、解析、链接、代码生成逻辑。

开发成本很高，而且可能被动陷入封闭的风险，但收益也是巨大的，它可以一路贯彻原则，以性能为最高优先级定制编译的各个阶段，比如说：

- 重写 ts 转译工具，完全抛弃 ts 类型检查，只做代码转换；
- 大多数打包工具把词法分析、语法分析、符号声明等步骤拆解为多个高内聚低耦合的处理单元，各个模块职责分明，可读性、可维护性较高。而 Esbuild 则坚持性能第一原则，不惜采用反直觉的设计模式，将多个处理算法混合在一起降低编译过程数据流转所带来的性能损耗；
- 一致的数据结构，以及衍生出的高效缓存策略；

这种深度定制一方面降低了设计成本，能够保持编译链条的架构一致性；一方面能够贯彻性能第一的原则，确保每个环节以及环节之间交互性能的最优。虽然伴随着功能、可读性、可维护性层面的牺牲，但在编译性能方面几乎做到了极致。

4. 结构一致性

在 Webpack 中使用 babel-loader 处理 JavaScript 代码时，可能需要经过多次数据转换：

- Webpack 读入源码，此时为字符串形式
- Babel 解析源码，转换为 AST 形式
- Babel 将源码 AST 转换为低版本 AST
- Babel 将低版本 AST generate 为低版本源码，字符串形式
- Webpack 解析低版本源码
- Webpack 将多个模块打包成最终产物

源码需要经历 `string => AST => AST => string => AST => string`，在字符串与 AST 之间反复横跳。

而 Esbuild 重写大多数转译工具之后，能够在多个编译阶段共用相似的 AST 结构，尽可能减少字符串到 AST 的结构转换，提升内存使用效率。

但与完全重写相对的是，目前Esbuild对于很多功能（像Vue、Angular等）支持还在逐步实现中，所以在线上环境还为时过早，但就纯编译性能上看，Esbuild可以极具竞争力，这也是为什么像Vite、snowpack选择Esbuild编译的原因。

但看官网作者貌似很坚持自己的想法，对于降级ES5、模块联邦、Vue、HMR等都不打算添加，所以线上环境还是建议使用基于Esbuild的上层框架，如Vite等

Upcoming roadmap

These features are already in progress and are first priority:

- Code splitting ([#16, docs](#))
- CSS content type ([#20, docs](#))
- Plugin API ([#111](#))

These are potential future features but may not happen or may happen to a more limited extent:

- HTML content type ([#31](#))
- Lowering to ES5 ([#297](#))
- Bundling top-level await ([#253](#))

After that point, I will consider esbuild to be relatively complete. I'm planning for esbuild to reach a mostly stable state and then stop accumulating more features. This will involve saying "no" to requests for adding major features to esbuild itself. I don't think esbuild should become an all-in-one solution for all frontend needs. In particular, I want to avoid the pain and problems of the "webpack config" model where the underlying tool is too flexible and usability suffers.

For example I am *not* planning to include these features in esbuild's core itself:

- Support for other frontend languages (e.g. [Elm](#), [Svelte](#), [Vue](#), [Angular](#))
- TypeScript type checking (just run `tsc` separately)
- An API for custom AST manipulation
- Hot-module reloading
- Module federation

I hope that the extensibility points I'm adding to esbuild ([plugins](#) and the [API](#)) will make esbuild useful to include as part of more customized build workflows, but I'm not intending or expecting these extensibility points to cover all use cases. If you have very custom requirements then you should be using other tools. I also hope esbuild inspires other build tools to dramatically improve performance by overhauling their implementations so that everyone can benefit, not just those that use esbuild.

4.4.2.2. Esbuild使用

1. Esbuild的特性

- 极快的速度，无需缓存；
- 支持 ES6 和 CommonJS 模块；
- 支持对 ES6 模块进行 tree shaking；
- [API](#) 可同时用于 JavaScript 和 Go；
- 兼容 [TypeScript](#) 和 [JSX](#) 语法；
- 支持 [Source maps](#)；

- 支持 [Minification](#);
- 支持 [plugins](#);

2. Esbuild的官方文档中也阐述了在Esbuild中API的使用, 针对不同类型文件的loader配置, plugins的基本使用, 内容不多, 建议直接看[官方](#)

不得不说Esbuild plugin的配置钩子相当简单, 一共只有四个: onResolve、onLoad、onStart、onEnd, 相对于webpack简直不要太舒服, 也让我们在自己写plugins或者用别人的plugins时能更快的使用

3. example

- [codesandbox](#);

4. 市场

- [awesome Esbuild](#);
- [esbuild plugins](#);

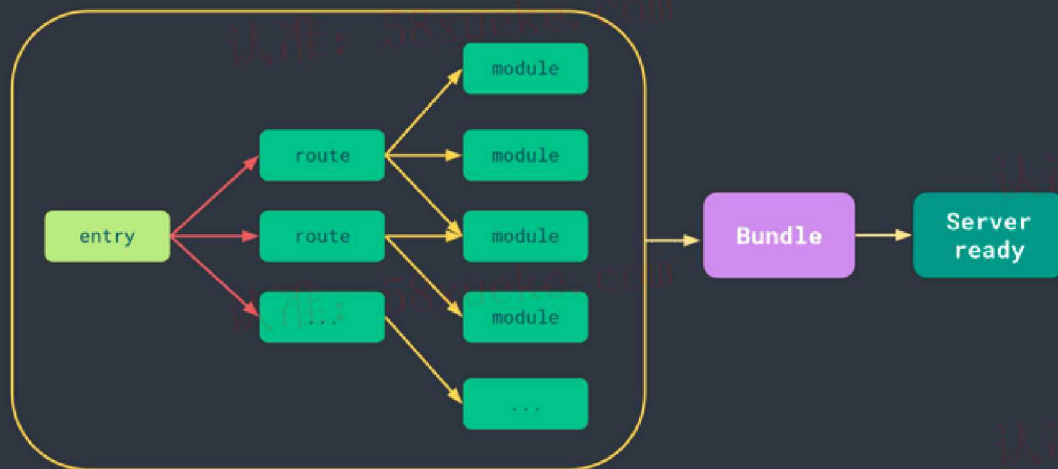
4.5. 基于ES Module的bundleless(no bundle)构建工具

browserify、webpack、rollup、parcel这些工具的思想都是递归循环依赖, 然后组装成依赖树, 优化完依赖树后生成代码。

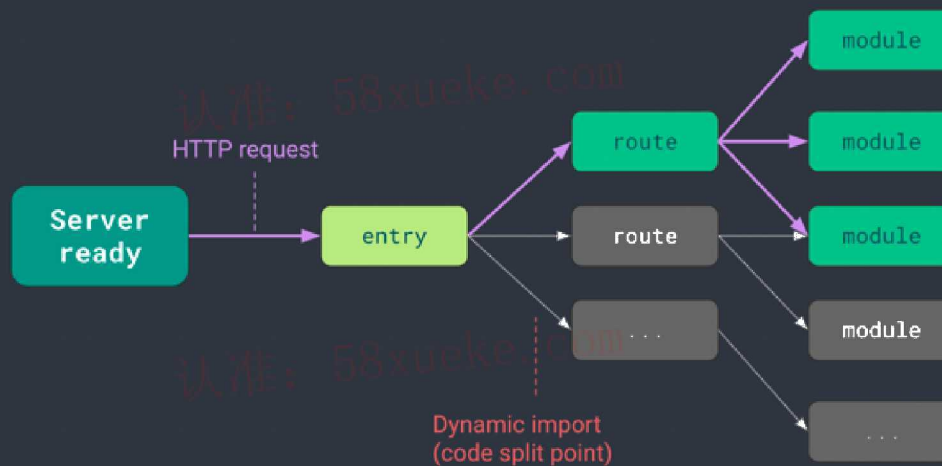
但是这样做的缺点就是慢, 需要遍历完所有依赖, 即使 parcel 利用了多核, webpack 也支持多线程, 在打包大型项目的时候依然慢可能会用上几分钟, 存在性能瓶颈。

所以基于浏览器原生 ESM 的运行时打包工具出现:

Bundle based dev server



Native ESM based dev server



可以看到，我们只需要打包当前所需要的资源，对于而不用打包整个项目，开发时的体验相比于 bundle 类的工具只能用极速来形容。bundleless 类运行时打包工具的启动速度是毫秒级的，因为不需要打包任何内容，只需要起两个 server，一个用于页面加载，另一个用于 HMR 的 WebSocket，当浏览器发出原生的 ES module 请求，server 收到请求只需编译当前文件后返回给浏览器不需要管依赖。

4.5.1. bundleless 可能性分析

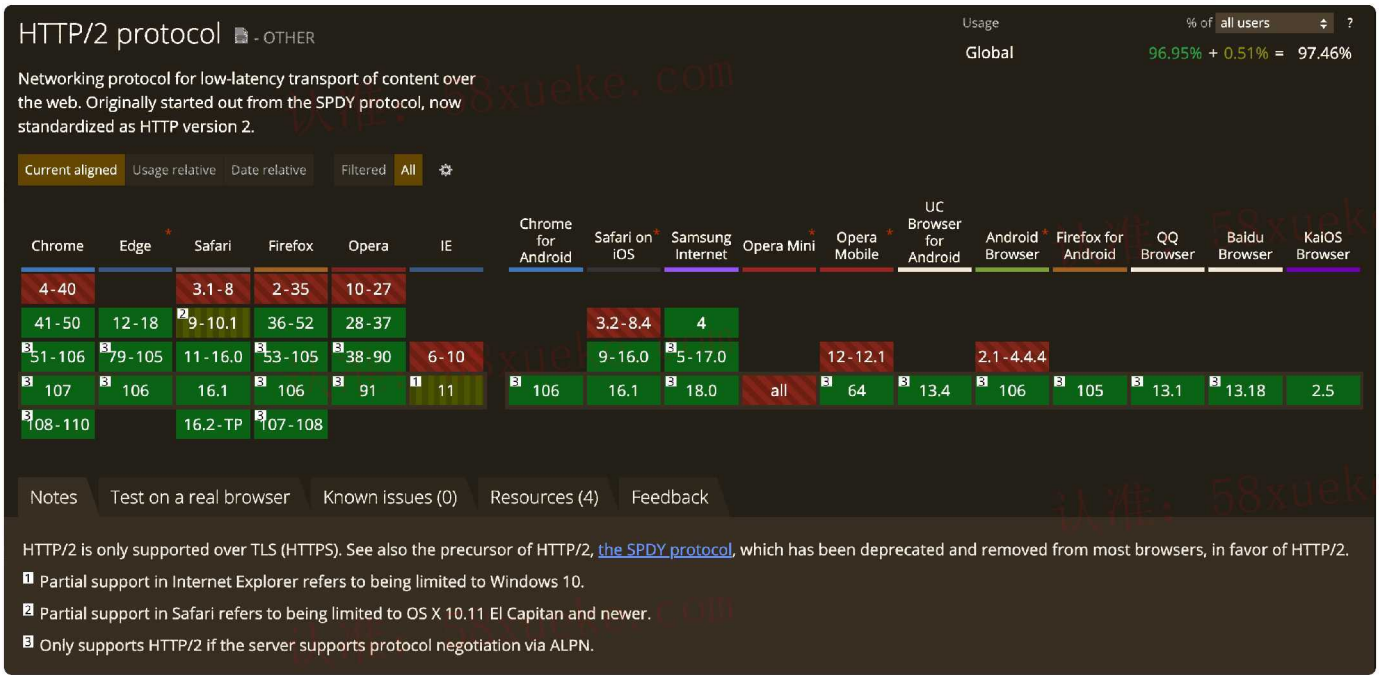
至于为什么 bundleless 在近几年很出名，原因在于：

4.5.1.1 HTTP2

因为http1.x不支持多路服用， HTTP 1.x 中，如果想并发多个请求，必须使用多个 TCP 链接，且浏览器为了控制资源，还会对单个域名有 6–8个的TCP链接请求的限制。因此我们需要做的就是将同域的一些静态资源比如js等，做一个资源合并，将多次请求不同的js文件，合并成单次请求一个合并后的大js文件。其实这也就是webpack的bundle由来。

而HTTP2实现了TCP链接的多路复用，因此同域名下不再有请求并发数的限制，我们可以同时请求同域名的多个资源，这个并发数可以很大，比如并发10， 50， 100个请求同时去请求同一个服务下的多个资源。（虽然实际效果有待考量），因为http2实现了多路复用，因此一定程度上，将多个静态文件打包到一起，从而减少请求次数，就不是必须的。

主流浏览器对HTTP2的支持情况如下：



除了IE以外，大部分浏览器对HTTP2的支持程度都很好，所以如果不用考虑兼容IE低版本，同时也不需要兼容低版本浏览器，不需要考虑不支持HTTP2的场景，所以在此情况下，让我们在使用bundleless上成为了可能。

4.5.1.2. 浏览器ESM

先来简单看下ESM的代码例子：

```

1 //main.j
2 import a from 'a.js'
3 console.log(a)
4 //a.js
5 export let a = 1

```

上述的ESM就是我们经常在项目中使用的ESM，在支持es6的浏览器中是可以直接使用的。

```

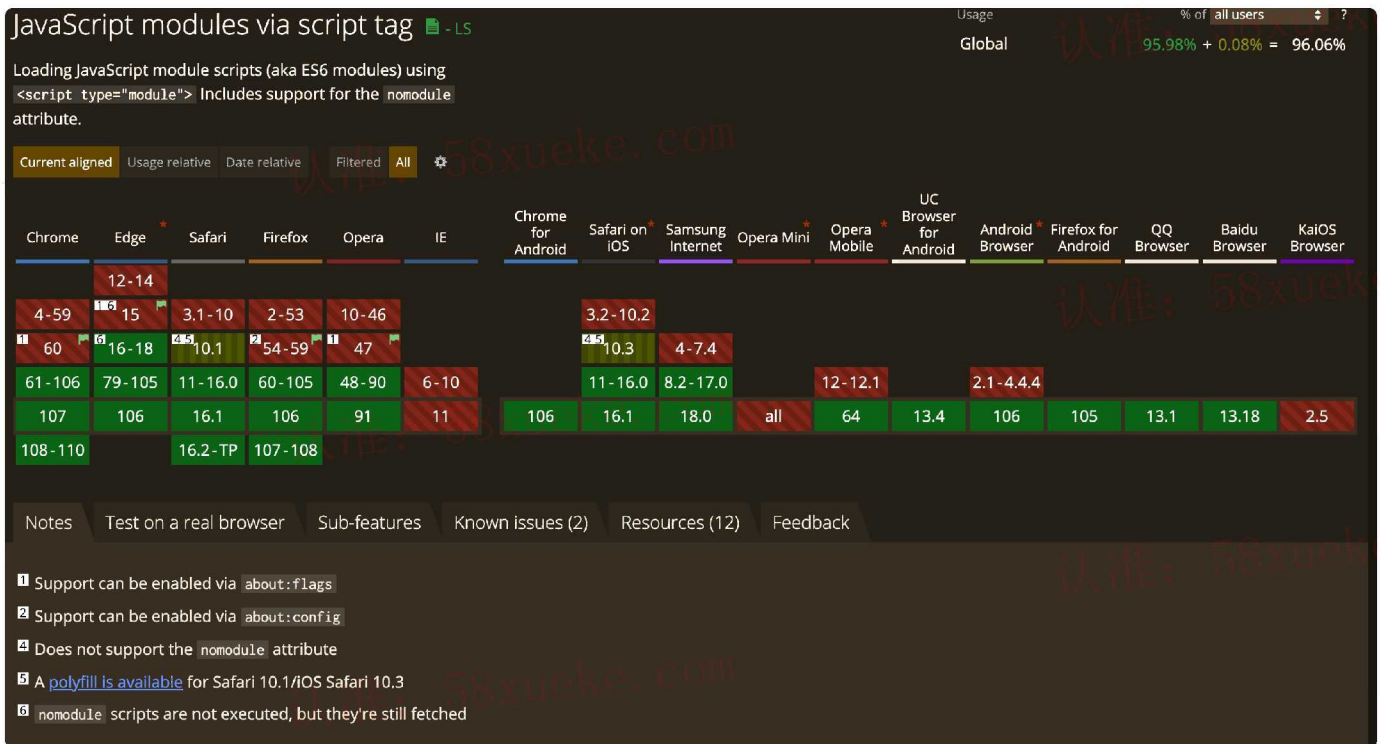
1 <html lang="en">
2   <body>
3     <div id="container">my name is {name}</div>
4     <script type="module">
5       import Vue from 'https://cdn.jsdelivr.net/npm/vue@2.6.12/dist/v
ue.esm.browser.js'
6       new Vue({
7         el: '#container',
8         data:{
9           name: 'Bob'
10        }
11      })
12    </script>
13  </body>
14 </html>

```

上述的代码中我们直接可以运行，我们根据script的 type="module" 可以判断浏览器支不支持ESM，如果不支持，该script里面的内容就不会运行。

首先我们来看主流浏览器对于ESM的支持情况：





从上图可以看出，主流的Edge, Chrome, Safari, and Firefox (+60)等浏览器都已经开始支持ESM。

4.5.1.3. 总结

浏览器对于HTTP2和ESM的支持，使得我们可以减少模块的合并，以及减少对于js模块化的处理。

- 如果浏览器支持HTTP2，那么一定程度上，我们不需要合并静态资源；
- 如果浏览器支持ESM，那么我们就不需要通过构建工具去维护复杂的模块依赖和加载关系；

这两点正是webpack等打包工具在bundle的时候所做的事情。浏览器对于HTTP2和ESM的支持使得我们减少bundle代码的场景。

4.5.2. Snowpack

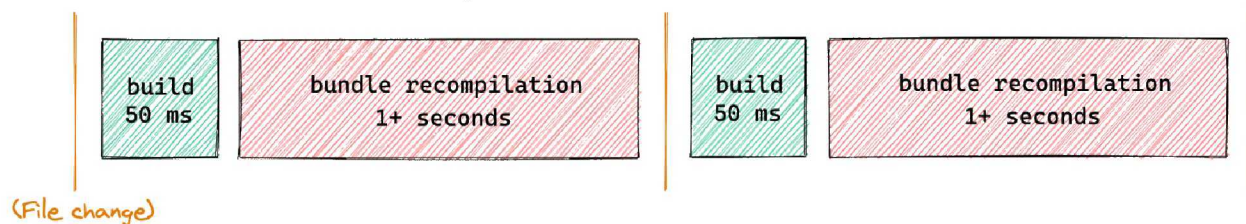
4.5.2.1. Snowpack介绍

Snowpack 是一个用于提升 Web 开发效率的轻量级新型构建工具。

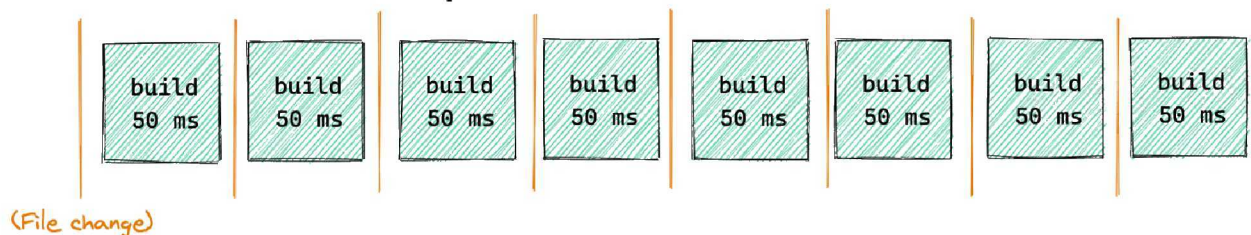
在开发过程中，Snowpack 为你的项目提供了免打包式(unbundled development) 的服务。每个文件只需构建一次就被永远缓存起来。当文件发生变化时，Snowpack 重新构建发生变化的文件然后在浏览器中直接更新，而没有在重新打包上浪费时间(通过模块热替换(HMR)实现)。

Snowpack 为你带来了两全其美的效果: 快速、免打包式的开发，以及打包式生产构建中的优化性能。

Bundled (ex: Webpack)



Unbundled (Snowpack)



针对npm依赖，NPM包主要是使用模块语法（Common.js，或CJS）发布的，如果没有一些构建处理，就不能在浏览器上运行。虽然用浏览器原生的ESMimport和export语句编写的代码会直接在浏览器中运行，但在导入npm包后你都会退回到打包式开发时代。

Snowpack采取了一种不同的方法：Snowpack没有因为这个打包整个应用程序，而是单独处理npm依赖。以下是它的工作原理。

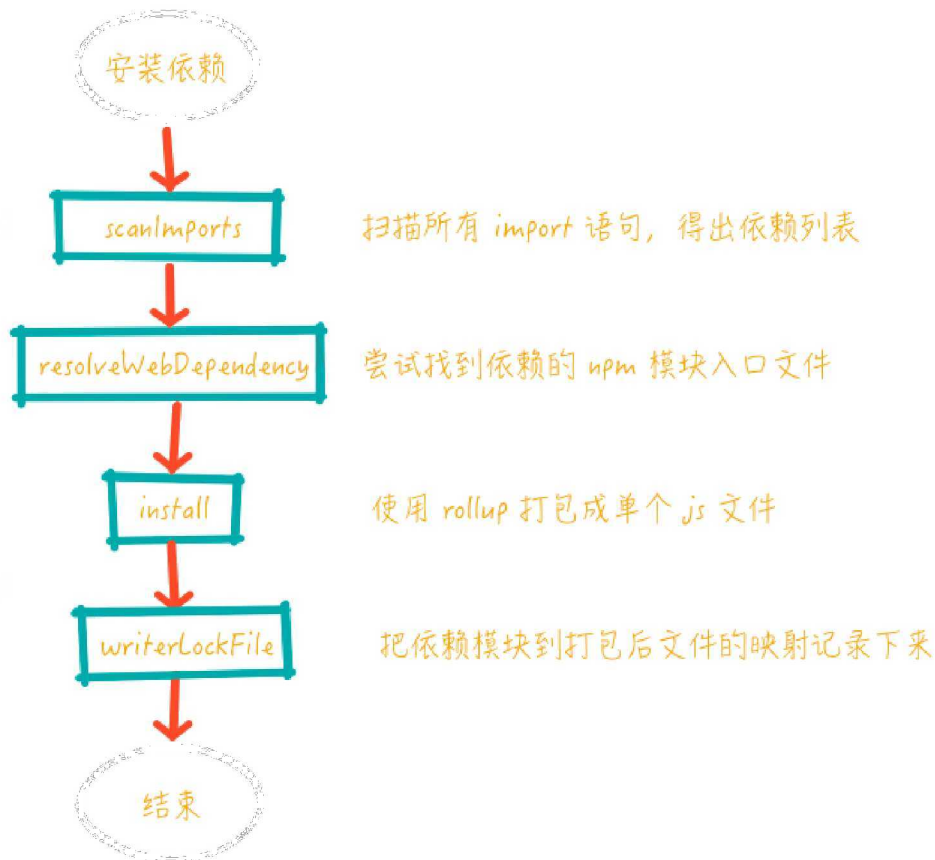
 复制代码

```
1 node_modules/react/**/* -> http://localhost:3000/web_modules/react.js
2 node_modules/react-dom/**/* -> http://localhost:3000/web_modules/react-dom.js
```

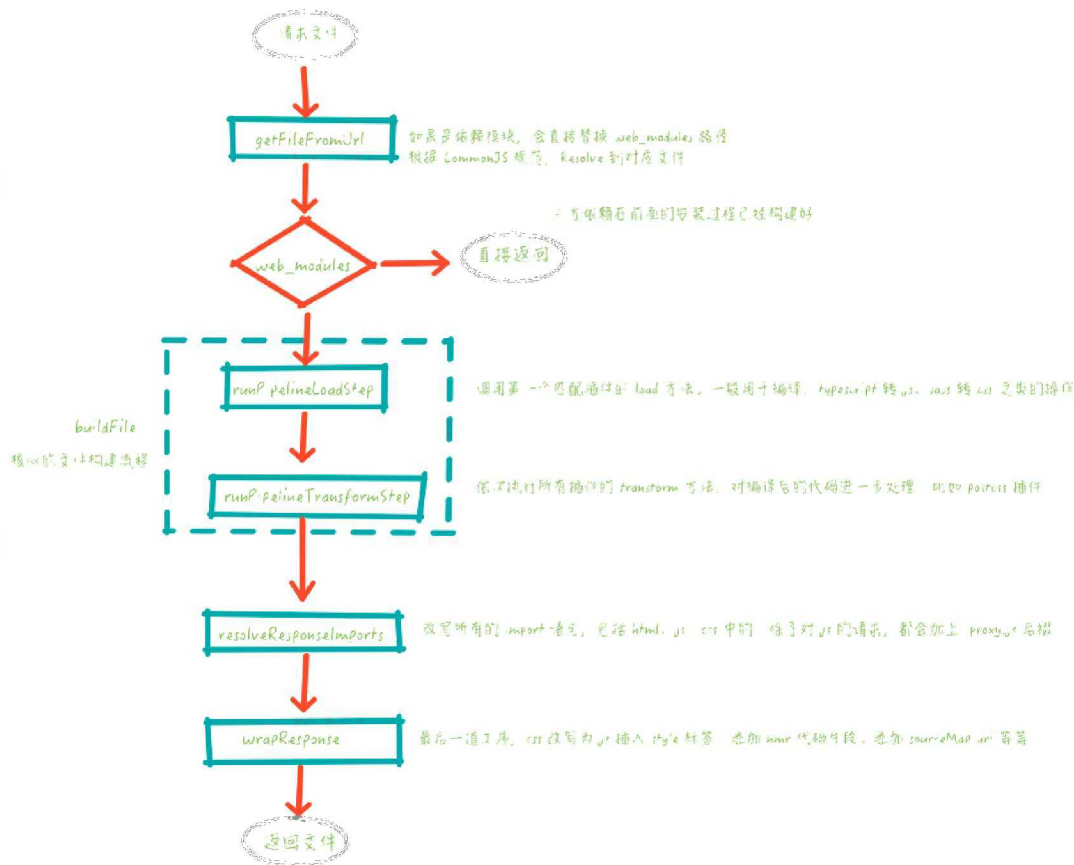
1. Snowpack 扫描网站/应用程序引入的所有 npm 包；
2. Snowpack 从node_modules目录中读取这些已安装的依赖包；
3. Snowpack 将所有 npm 依赖分别打包到单个 JavaScript 文件中，例如：react和react-dom分别转换为react.js和react-dom.js；
4. 每个转换来的文件在经过 ESM 的import语句导入后，都可以直接在浏览器中运行；
5. 因为 npm 依赖很少改变，Snowpack 很少需要重建它们；

在 Snowpack 执行完对 npm 依赖的处理后，任何包都可以被导入并直接在浏览器中运行，不需要额外的打包或工具。这种在浏览器中原生导入 npm 包的能力（无需打包器）是所有免打包式开发工具和 Snowpack 建立的基础。

4.5.2.2. Snowpack原理



1. snowpack 会扫描工作目录下所有源码，得出所有的依赖列表。具体来说，对于 css、less、sass、scss 文件，会扫描所有的 @import 语句，对于 html、svelte、vue 文件，会扫描所有 <script> 标签内的 import 语句；对于 js、jsx、mjs、ts、tsx 文件，会解析所有的 import 语句，得到依赖的 npm 模块名称。
2. 接下来，根据依赖模块名称，尝试去找 npm 模块入口文件。采用的策略和 node 依赖查找机制类似，找到 package.json 文件后，先找 export map 指定的入口文件，再依次找 browser:module、module、main:esnext、browser、main 等字段，保证优先使用 ESM 入口。
3. 使用 rollup 打包，将依赖的模块打包成一个个 ESM，从而可以直接在浏览器运行，输出到缓存目录（默认是 ./node_modules/.cache/snowpack/dev）下。这一步打包操作也可以同时避免 ESM 依赖地狱的问题。（试想一下，如果不对依赖模块提前打包，而是通过一个个 HTTP 请求来加载 node_modules 下所有模块的场景）
4. 最后把依赖的模块与打包后文件绝对路径的映射记录下来，后面构建文件阶段根据这个映射对业务代码中原有的 import 语句进行改写。



首先，如果是对依赖模块的请求，会直接返回之前构建好的 ESM。

接下来，对于业务代码的处理，则会先经过文件的编译过程。这一过程是由不同的插件来实现，分为 load 和 transform 两个阶段。load 阶段是将业务代码编译成为浏览器可以直接运行的代码，比如 TypeScript、JSX 到 JS，Sass 到 CSS。transform 是对编译后代码做进一步的处理，典型的比如 PostCss。

4.5.2.3. 使用esbuild

mjs、jsx、ts、tsx 这几种格式的脚本文件浏览器是无法直接执行的，为此，snowpack 内置了一个 esbuild 插件，如果用户没有显式指定用于处理 mjs、jsx、ts、tsx 的插件，那么这个内置的插件就会生效，用 esbuild 进行这几类文件的编译，从而默认支持 TypeScript、JSX：

```
1 // add internal JS handler plugin if none specified
2 const needsDefaultPlugin = new Set(['.mjs', '.jsx', '.ts', '.tsx']);
3 plugins
4   .filter(({resolve}) => !!resolve)
5   .reduce((arr, a) => arr.concat(a.resolve!.input), [] as string[])
6   .forEach((ext) => needsDefaultPlugin.delete(ext));
7 if (needsDefaultPlugin.size > 0) {
8   plugins.unshift(execPluginFactory(esbuildPlugin, {input: [...needsDefault
    Plugin]}));
9 }
```

esbuild 插件的实现，其实就是在 load 阶段使用 esbuild 进行编译：



```

1  export function esbuildPlugin(config: SnowpackConfig, {input}: {input: string[]}): SnowpackPlugin {
2    return {
3      name: '@snowpack/plugin-esbuild',
4      resolve: {
5        input,
6        output: ['.js'],
7      },
8      async load({filePath}) {
9        esbuildService = esbuildService || (await startService());
10       const contents = await fs.readFile(filePath, 'utf-8');
11       const isPreact = checkIsPreact(filePath, contents);
12       const {js, jsSourceMap, warnings} = await esbuildService!.transform(
13         contents, {
14           loader: getLoader(filePath),
15           jsxFactory: isPreact ? 'h' : undefined,
16           jsxFragment: isPreact ? 'Fragment' : undefined,
17           sourcefile: filePath,
18           sourcemap: config.buildOptions.sourceMaps,
19         });
20       return {
21         '.js': {
22           code: js || '',
23           map: jsSourceMap,
24         },
25       },
26       cleanup() {
27         esbuildService && esbuildService.stop();
28       },
29     };
30   }

```

- example: [codesandbox](#);
- 市场: [snowpack](#) ;

4.5.3. Vite

下一代前端构建工具

4.5.3.1. Vite介绍

Vite被广大开发者认为是下一代的构建工具，包含了这些特点：

- Instant Server Start —— 即时服务启动
- Lightning Fast HMR —— 闪电般快速的热更新
- Rich Features —— 丰富的功能
- Optimized Build —— 经过优化的构建
- Universal Plugin Interface —— 通用的Plugin接口
- Fully Typed APIs —— 类型齐全的API

针对以往的打包构建工具（如webpack），Vite做到了：

1. 开发环境冷启动无需打包

解决了启动慢的问题，无需分析模块之间的依赖，同时也无需在启动开发服务器前进行编译，启动时还会使用esbuild来进行预构建。而Webpack启动后会做一堆事情，经历一条很长的编译打包链条，从入口开始需要逐步经历语法解析、依赖收集、代码转译、打包合并、代码优化，最终将高版本的、离散的源码编译打包成低版本、高兼容性的产物代码；

2. 优化HMR

针对HMR慢，即使只有很小的改动，Webpack依然需要构建完整的模块依赖图，并根据依赖图来进行转换。而Vite利用了ESM和浏览器缓存技术，更新速度与项目复杂度无关。如Snowpack、Vite这类面向非打包的构建工具，在开发环境启动时只需要启动两个Server，一个用于页面加载，一个用于HMR的Websocket。当浏览器发出原生的ESM请求，Server收到请求只需要编译当前文件后返回给浏览器，不需要管理依赖。

3. 使用简单，开箱即用

相比Webpack需要对entry、loader、plugin等进行诸多配置，Vite的使用可谓是相当简单了。只需执行初始化命令，就可以得到一个预设好的开发环境，开箱即获得一堆功能，包括：CSS预处理、html预处理、异步加载、分包、压缩、HMR等。他使用复杂度介于Parcel和Webpack的中间，只是暴露了极少数的配置项和plugin接口，既不会像Parcel一样配置不灵活，又不会像Webpack一样需要了解庞大的loader、plugin生态，灵活适中、复杂度适中。适合前端新手。

4.5.3.2. Vite原理

开发环境&生产环境

- 开发环境：

不需要对所有资源打包，只是使用esbuild对依赖进行预构建，将CommonJS和UMD发布的依赖转换为浏览器支持的ESM，同时提高了后续页面的加载性能（lodash的请求）。Vite会将于构建的依赖缓存到node_modules/.vite目录下，它会根据几个源来决定是否需要重新运行预构建，包括 packages.json中

的dependencies列表、包管理器的lockfile、可能在vite.config.js相关字段中配置过的。只要三者之一发生改变，才会重新预构建；

同时，开发环境使用了浏览器缓存技术，解析后的依赖请求以http头的 max-age=31536000, immutable 强缓存，以提高页面性能；

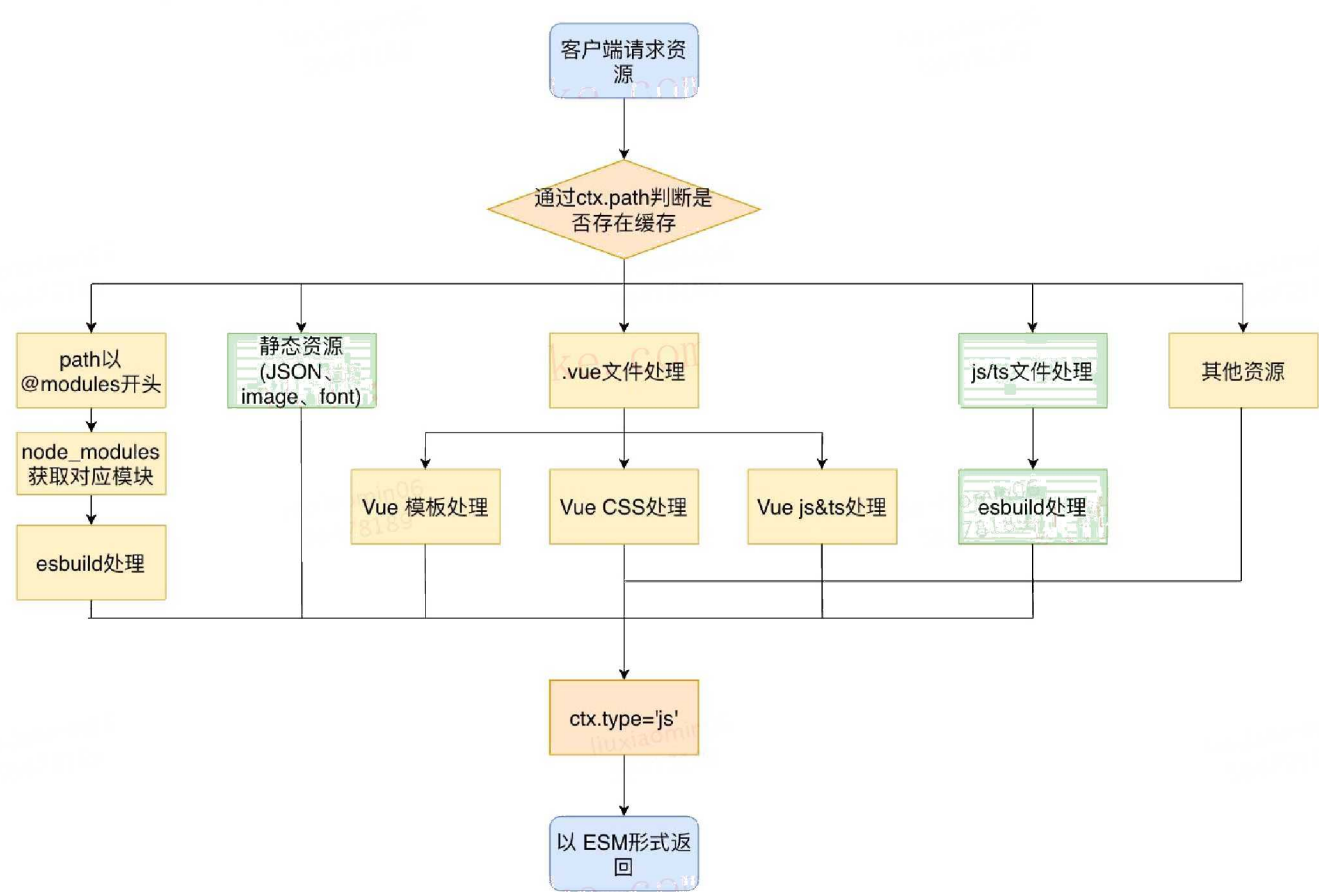
- 生产环境

由于嵌套导入会导致发送大量的网络请求，即使使用HTTP2.x（多路复用、首部压缩），在生产环境中发布未打包的ESM仍然性能低下。因此，对比在开发环境Vite使用esbuild来构建依赖，生产环境Vite则使用了更加成熟的Rollup来完成整个打包过程。因为esbuild虽然快，但针对应用级别的代码分割、CSS处理仍然不够稳定，同时也未能兼容一些未提供ESM的SDK。

为了在生产环境中获得最佳的加载性能，仍然需要对代码进行tree-shaking、懒加载以及chunk分割（以获得更好的缓存）。

请求拦截

启动一个 koa 服务器拦截由浏览器请求 ESM的请求。通过请求的路径找到目录下对应的文件做一定的处理最终以 ESM的格式返回给客户端。



浏览器对 import 的模块发起请求时的一些局限了，平时我们写代码，如果不是引用相对路径的模块，而是引用 node_modules 的模块，都是直接 import xxx from 'xxx'，由 Webpack 等工具来帮我

们找这个模块的具体路径。但是浏览器不知道你项目里有 `node_modules`，它只能通过相对路径去寻找模块。

因此 Vite 在拦截的请求里，对直接引用 `node_modules` 的模块都做了路径的替换，换成了 `/@modules/` 并返回回去。而后浏览器收到后，会发起对 `/@modules/xxx` 的请求，然后被 Vite 再次拦截，并由 Vite 内部去访问真正的模块，并将得到的内容再次做同样的处理后，返回给浏览器。

普通 JS import 替换

```

1 // src/node/serverPluginModuleRewrite.ts
2
3 // 只取关键代码:
4 // Vite 使用 Koa 作为内置的服务器
5 // 如果请求的路径是 /index.html
6 if (ctx.path === '/index.html') {
7   // ...
8   const html = await readBody(ctx.body)
9   ctx.body = html.replace(
10     /(<script\b[^(>]*>)([\\s\\S]*?)<\/script>/gm, // 正则匹配
11     (_, openTag, script) => {
12       // also inject __DEV__ flag
13       const devFlag = hasInjectedDevFlag ? `` : devInjectionCode
14       hasInjectedDevFlag = true
15       // 替换 html 的 import 路径
16       return `${devFlag}${openTag}${rewriteImports(
17         script,
18         '/index.html',
19         resolver
20       )}<\/script>`
21     }
22   )
23   // ...
24 }
25
26 // 如果并没有在 script 标签内部直接写 import, 而是用 src 的形式引用的话如下:
27
28 <script type="module" src="/main.js"><\/script>
29
30
31 // 只取关键代码:
32 if (
33   ctx.response.is('js') &&
34   // ...
35 ) {
36   // ...
37   const content = await readBody(ctx.body)
38   await initLexer
39   // 重写 js 文件里的 import
40   ctx.body = rewriteImports(
41     content,
42     ctx.url.replace(/(&|\\?)t=\\d+/, ''),
43     resolver,
44     ctx.query.t
45   )

```



```
46 // 写入缓存，之后可以从缓存中直接读取
47 rewriteCache.set(content, ctx.body)
48 }
49
50 // 其中的替换逻辑用的是 es-module-lexer 来进行的语法分析获取 imports 数组，然后再做的替换。
```

*.vue 文件的替换

把原本一个 .vue 的文件拆成了三个请求（分别对应 script、style 和 template），浏览器会先收到包含 script 逻辑的 App.vue 的响应，然后解析到 template 和 style 的路径后，会再次发起 HTTP 请求来请求对应的资源，此时 Vite 对其拦截并再次处理后返回相应的内容。



```
1 // src/node/serverPluginVue.ts
2 // 核心逻辑是根据 URL 的 query 参数来做不同的处理（简化分析如下）：
3
4 // 如果没有 query 的 type, 比如直接请求的 /App.vue
5 if (!query.type) {
6   ctx.type = 'js'
7   ctx.body = compileSFCMain(descriptor, filePath, publicPath) // 编译 App.v
  ue, 编译成上面说的带有 script 内容, 以及 template 和 style 链接的形式。
8   return etagCacheCheck(ctx) // ETAG 缓存检测相关逻辑
9 }
10
11 // 如果 query 的 type 是 template, 比如 /App.vue?type=template&xxx
12 if (query.type === 'template') {
13   ctx.type = 'js'
14   ctx.body = compileSFCTemplate( // 编译 template 生成 render function
15     // ...
16   )
17   return etagCacheCheck(ctx)
18 }
19
20 // 如果 query 的 type 是 style, 比如 /App.vue?type=style&xxx
21 if (query.type === 'style') {
22   const index = Number(query.index)
23   const styleBlock = descriptor.styles[index]
24   const result = await compileSFCStyle( // 编译 style
25     // ...
26   )
27   if (query.module !== null) { // 如果是 css module
28     ctx.type = 'js'
29     ctx.body = `export default ${JSON.stringify(result.modules)}`
30   } else { // 正常 css
31     ctx.type = 'css'
32     ctx.body = result.code
33   }
34 }
```

@modules/* 路径解析

```

1 // 解析的逻辑来自 src/node/serverPluginModuleResolve.ts。
2 // node_modules 里找有没有对应的模块，有的话就返回，没有的话就报 404:
3 // ...
4 try {
5   const file = resolve(root, id) // id 是模块的名字，比如 axios
6   return serve(id, file, 'node_modules') // 从 node_modules 中找到真正的模块
    内容并返回
7 } catch (e) {
8   console.error(
9     chalk.red(`[vite] Error while resolving node_modules with id "${id}":`)
10  )
11   console.error(e)
12   ctx.status = 404 // 如果没找到就 404
13 }

```

热更新原理

Vite 的热更新就是在客户端与服务端建立了一个 websocket 连接，当代码被修改时，服务端发送消息通知客户端去请求修改模块的代码，完成热更新。

- 服务端：服务端做的就是监听代码文件的改变，在合适的时机向客户端发送 websocket 信息通知客户端去请求新的模块代码。
- 客户端：Vite 中客户端的 websocket 相关代码在处理 html 中时被写入代码中。可以看到在处理 html 时，vite/client 的相关代码已经被插入。

```

1 export const clientPublicPath = `/vite/client`
2 const devInjectionCode = `<script type="module">import "${clientPublicPath}"</script>`
3 async function rewriteHtml(importer: string, html: string) {
4   return injectScriptToHtml(html, devInjectionCode)
5 }

```

当 request.path 路径是 /vite/client 时，请求获取已经提前写好的关于 websocket 的代码。因此在客户端中我们创建了一个 websocket 服务并与服务端建立了连接。

Vite 会接受来自客户端的消息。通过不同的消息触发一些事件。做到浏览器端的即时热模块更换（热更新）。包括 connect、vue-reload、vue-rerender 等事件，分别触发组件 vue 的重新加载，render 等。

```

1 // Listen for messages
2 socket.addEventListener('message', async ({ data }) => {
3   const payload = JSON.parse(data) as HMRPayload | MultiUpdatePayload
4   if (payload.type === 'multi') {
5     payload.updates.forEach(handleMessage)
6   } else {
7     handleMessage(payload)
8   }
9 })
10
11 async function handleMessage(payload: HMRPayload) {
12   const { path, changeSrcPath, timestamp } = payload as UpdatePayload
13   console.log(path)
14   switch (payload.type) {
15     case 'connected':
16       console.log(`[vite] connected.`)
17       break
18     case 'vue-reload':
19       queueUpdate(
20         import(`${path}?t=${timestamp}`)
21           .catch((err) => warnFailedFetch(err, path))
22           .then((m) => () => {
23             __VUE_HMR_RUNTIME__.reload(path, m.default)
24             console.log(`[vite] ${path} reloaded.`)
25           })
26       )
27       break
28     case 'vue-rerender':
29       const templatePath = `${path}?type=template`
30       import(`${templatePath}&t=${timestamp}`).then((m) => {
31         __VUE_HMR_RUNTIME__.rerender(path, m.render)
32         console.log(`[vite] ${path} template updated.`)
33       })
34       break
35     case 'style-update':
36       // check if this is referenced in html via <link>
37       const el = document.querySelector(`link[href*='${path}']`)
38       if (el) {
39         el.setAttribute(
40           'href',
41           `${path}${path.includes('?') ? '&' : '?'}t=${timestamp}`
42         )
43         break
44       }
45       const importQuery = path.includes('?') ? '&import' : '?import'

```

```

46     await import(`${path}${importQuery}&t=${timestamp}`)
47     console.log(`[vite] ${path} updated.`)
48     break
49   case 'js-update':
50     queueUpdate(updateModule(path, changeSrcPath, timestamp))
51     break
52   case 'custom':
53     const cbs = customUpdateMap.get(payload.id)
54     if (cbs) {
55       cbs.forEach((cb) => cb(payload.customData))
56     }
57     break
58   case 'full-reload':
59     if (path.endsWith('.html')) {
60       // if html file is edited, only reload the page if the browser is
61       // currently on that page.
62       const pagePath = location.pathname
63       if (
64         pagePath === path ||
65         (pagePath.endsWith('/') && pagePath + 'index.html' === path)
66       ) {
67         location.reload()
68       }
69       return
70     } else {
71       location.reload()
72     }
73   }
74 }

```